Trabalho Prático 3 — sched_profiler

O que o programa faz

O sched_profiler cria a quantidade de threads informada. Cada uma recebe uma letra e uma configuração própria, no formato SCHED_POLITICA:prioridade. A primeira é A, a segunda B e assim por diante. O programa aceita de 1 a 26 threads. Todos os testes abaixo usaram quatro políticas: SCHED_IDLE, SCHED_LOW_IDLE, SCHED_FIFO e SCHED_RR.

As threads escrevem em um buffer global, uma letra por posição. O semáforo começa em zero para que a função principal configure todas as threads antes de liberar a escrita. Em seguida, cada thread espera o semáforo, confere se há espaço, escreve a letra, avança o ponteiro compartilhado, atualiza sua contagem e libera o semáforo. Assim, duas threads não modificam a mesma posição ao mesmo tempo.

No final, o programa imprime o buffer completo, resume sequências de letras repetidas em blocos e mostra, por thread, política, prioridade, blocos, caracteres preenchidos e escalonamentos. A última métrica vem do terceiro campo de /proc/thread-self/schedstat, lido pela própria thread antes de terminar. O kernel do experimento foi configurado com SCHEDSTATS.

Peso e prioridade nas políticas

No kernel usado nos testes, SCHED_IDLE mantém o peso 3 e SCHED_LOW_IDLE usa peso 1. No CFS, o peso influencia como o tempo virtual de uma thread avança: com peso menor, ele avança mais rapidamente pelo mesmo tempo de CPU, favorecendo a escolha das demais threads quando disputam a mesma CPU. O inverso do peso de LOW_IDLE foi ajustado para 4294967295U. Os valores 3 e 1 não garantem, por si só, uma proporção exata de caracteres escritos: o semáforo, outras tarefas e o número de CPUs também influenciam o resultado.

IDLE e LOW_IDLE usam prioridade numérica 0 na configuração das threads; o peso é uma informação diferente dessa prioridade. FIFO e RR pertencem à classe de tempo real, cuja disputa considera primeiro a prioridade numérica: 2 tem precedência sobre 1. Entre threads de mesma prioridade, FIFO e RR seguem regras diferentes de permanência e rodízio na CPU. Por isso, os pesos 3 e 1 não são usados para explicar FIFO e RR.

Blocos e escalonamentos não são a mesma contagem: uma thread pode voltar à CPU sem que outra tenha escrito no intervalo. Nesse caso há uma nova entrada na CPU, mas o buffer continua no mesmo bloco de letras.

Como os testes foram lidos

Todas as execuções preencheram 5.000.000 de posições. A ordem A, B, C, D corresponde à ordem dos argumentos. IDLE e LOW_IDLE usam prioridade numérica 0; em FIFO e RR, 2 é maior que 1. O trace foi gravado com sched:sched_switch e aberto no KernelShark. Os resumos abaixo trazem as contagens exibidas pelo programa; as imagens mostram a atividade das CPUs e as trocas de tarefas.

Resultados

Valores por thread: blocos / caracteres / escalonamentos. A, B, C e D seguem a ordem de criação.

Uma CPU

Caso	Thread: blocos / caracteres / escalonamentos
IDLE 0 × LOW_IDLE 0 2 threads	A: 108 / 3.186.188 / 238 B: 107 / 1.813.812 / 230
IDLE 0 × LOW_IDLE 0 4 threads (A,B IDLE; C,D LOW)	A: 107 / 1.700.393 / 249 B: 105 / 1.898.761 / 244 C: 75 / 651.954 / 160 D: 73 / 748.892 / 164
FIFO 1 × FIFO 2	A: 0 / 0 / 2 B: 1 / 5.000.000 / 7

RR 1 × RR 2	A: 0 / 0 / 2 B: 1 / 5.000.000 / 6
FIFO 1 × RR 1	A: 1 / 5.000.000 / 7 B: 0 / 0 / 2
FIFO 1 × RR 2	A: 0 / 0 / 2 B: 1 / 5.000.000 / 7
RR 1, quatro threads	A: 5 / 935.960 / 11 B: 5 / 2.151.475 / 10 C: 4 / 1.510.244 / 10 D: 4 / 402.321 / 12

Duas CPUs

Caso	Thread: blocos / caracteres / escalonamentos
IDLE 0 × LOW_IDLE 0 2 threads	A: 403.611 / 2.436.994 / 25.915 B: 403.612 / 2.563.006 / 14.257
IDLE 0 × LOW_IDLE 0 4 threads (A,B IDLE; C,D LOW)	A: 1.001.844 / 1.234.821 / 5.327 B: 1.885.144 / 2.403.576 / 2.903 C: 555.528 / 682.173 / 3.727 D: 558.097 / 679.430 / 3.870
FIFO 1 × FIFO 2	A: 416.686 / 2.434.225 / 22.347 B: 416.686 / 2.565.775 / 20.801
RR 1 × RR 2	A: 552.873 / 2.473.770 / 20.363 B: 552.874 / 2.526.230 / 19.245
RR 1, quatro threads	A: 1.105.671 / 1.232.432 / 1.206 B: 1.106.885 / 1.226.723 / 1.252 C: 1.161.885 / 1.265.905 / 1.380 D: 1.154.365 / 1.274.940 / 1.186

Três traces que explicam os resultados

1. IDLE e LOW_IDLE em uma CPU

Com uma thread de cada política, A (IDLE) escreveu 3.186.188 caracteres (63,7%) e B (LOW_IDLE), 1.813.812 (36,3%). As duas trabalharam, mas IDLE participou mais quando precisaram dividir a mesma CPU. O peso menor de LOW_IDLE é compatível com essa diferença. A quantidade de escalonamentos foi próxima, 238 contra 230: entrar na CPU um número parecido de vezes não significa escrever a mesma quantidade.

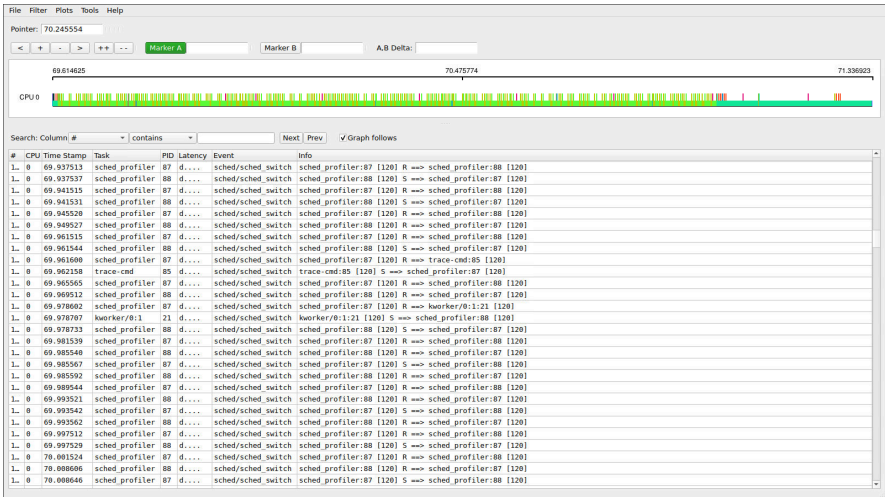


Figura 1 — IDLE × LOW_IDLE, duas threads em uma CPU

2. IDLE e LOW_IDLE em duas CPUs

Com duas CPUs e as mesmas duas threads, A escreveu 2.436.994 (48,7%) e B, 2.563.006 (51,3%). Ambas puderam executar em CPUs diferentes; o peso menor de LOW_IDLE deixou de produzir a vantagem observada para IDLE no teste de uma CPU. O gráfico mostra atividade do programa nas duas linhas de CPU. A diferença de escalonamentos, 25.915 contra 14.257, também não equivale à diferença de caracteres.

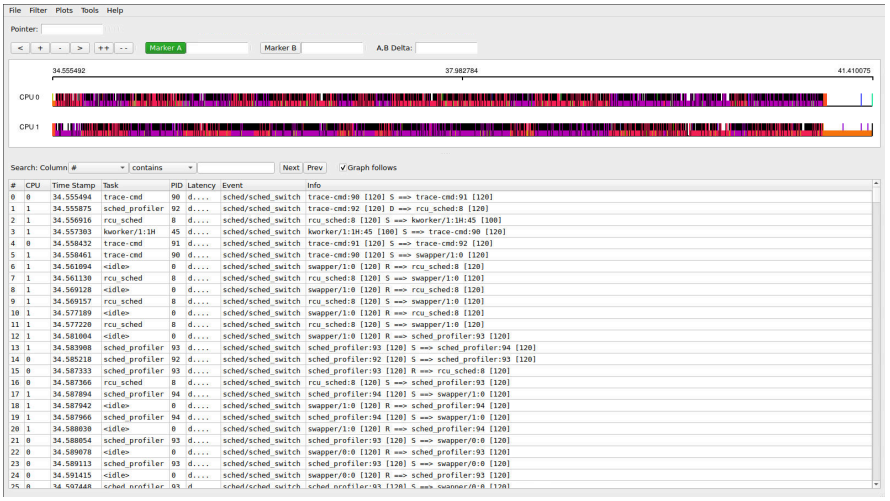


Figura 2 — IDLE × LOW_IDLE, duas threads em duas CPUs

3. FIFO 1 e RR 2 em uma CPU

Aqui A usou FIFO com prioridade 1 e B usou RR com prioridade 2. B escreveu todos os 5.000.000 de caracteres; A, nenhum. A comparação mostra o efeito da prioridade numérica maior: ela prevaleceu nesta disputa mesmo estando em RR. Este caso não isola a diferença entre as regras FIFO e RR, porque as prioridades são diferentes. A recebeu duas entradas na CPU, mas não encontrou uma oportunidade de escrever antes de o buffer encher.

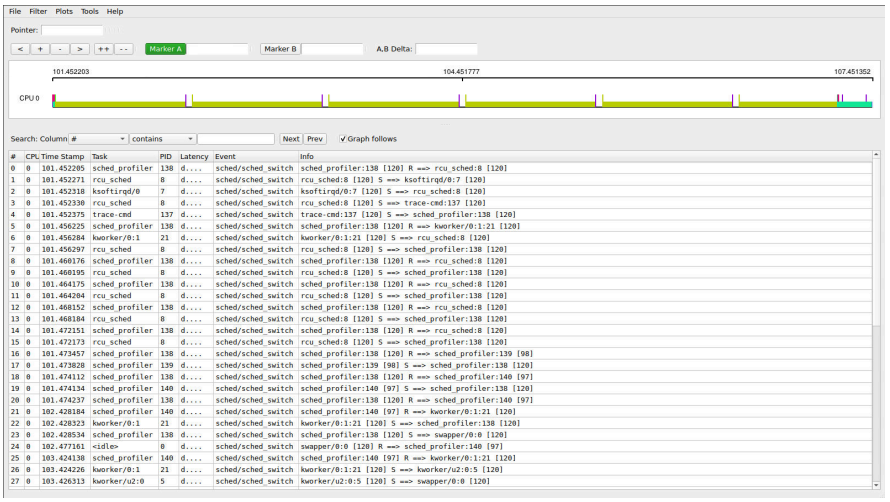


Figura 3 — FIFO prioridade 1 × RR prioridade 2 em uma CPU

O que mostram os outros nove testes

As capturas disponíveis dos demais testes estão na pasta capturas do arquivo ZIP. O teste RR com quatro threads em duas CPUs tem resultado registrado, mas não possui captura nessa pasta.

IDLE × LOW_IDLE, quatro threads, uma CPU. As duas IDLE somaram 3.599.154 caracteres (72,0%) e as duas LOW_IDLE, 1.400.846 (28,0%). O resultado reforça, nesta rodada, a maior participação de IDLE sob disputa na mesma CPU.

IDLE × LOW_IDLE, quatro threads, duas CPUs. As IDLE somaram 3.638.397 (72,8%) e as LOW_IDLE, 1.361.603 (27,2%). Com quatro threads para duas CPUs ainda havia disputa; nesta execução, as participações dos grupos ficaram diferentes, ao contrário do caso com duas threads.

FIFO 1 × FIFO 2, uma CPU. A thread de prioridade 2 escreveu o buffer inteiro; a de prioridade 1 escreveu zero. O número maior teve preferência enquanto ambas disputavam a única CPU.

FIFO 1 × FIFO 2, duas CPUs. As duas escreveram: 2.434.225 e 2.565.775 caracteres. Com uma CPU disponível para cada uma, a prioridade maior não excluiu a outra da execução.

RR 1 × RR 2, uma CPU. A thread RR de prioridade 2 escreveu todos os caracteres. Como as prioridades eram diferentes, a thread de prioridade 2 teve preferência; o rodízio entre threads RR ocorre quando elas têm a mesma prioridade.

RR 1 × RR 2, duas CPUs. As duas escreveram quase metade do buffer: 2.473.770 e 2.526.230. A prioridade diferente não impediu a execução simultânea em CPUs distintas.

FIFO 1 × RR 1, uma CPU. A FIFO escreveu os 5.000.000 de caracteres e a RR, zero. Com prioridade numérica igual, a thread FIFO que continuou pronta não cedeu a CPU para a RR nesta execução.

RR 1, quatro threads, uma CPU. Todas escreveram, mas em quantidades desiguais: de 402.321 a 2.151.475 caracteres. Uma rodada com o semáforo e execução curta não estabelece uma divisão igual de escritas.

RR 1, quatro threads, duas CPUs. Todas ficaram próximas de 1,25 milhão de caracteres, de 1.226.723 a 1.274.940. Nesta rodada, a participação foi mais equilibrada que no caso de uma CPU.

Conclusão

Com uma CPU, IDLE escreveu mais que LOW_IDLE nos testes realizados. Esse resultado é compatível com o peso maior de IDLE no CFS, embora a quantidade escrita também dependa do semáforo e das demais tarefas. Nas disputas entre prioridades diferentes de FIFO e RR, a thread de prioridade 2 preencheu o buffer. Com FIFO e RR na mesma prioridade, a FIFO preencheu o buffer na execução registrada.

Com duas CPUs e duas threads, ambas escreveram quantidades próximas, pois puderam executar em CPUs diferentes. Com quatro threads, a disputa continuou: no teste IDLE × LOW_IDLE, o grupo IDLE escreveu mais; no teste RR com prioridades iguais, as quatro ficaram próximas nesta rodada.

O buffer mostra a ordem e a quantidade de escritas. O schedstat registra entradas em execução, e o KernelShark mostra as trocas de tarefas ao longo do tempo. Como foi registrada uma execução por configuração, os resultados descrevem essas rodadas e podem variar em novas execuções.